Spec-Driven Development

With GitHub Spec-Kit

From blank page to finished draft — building a book-writing app, one spec at a time.

Barret Blake
Architect, NimblePros
https://barretblake.dev
BlueSky: @barretblake.dev
https://linkedin.com/in/barretblake







MOMENTUM
Developer Conference
powered by The Circuit

WHO AM I

Barret Blake



Barret Blake

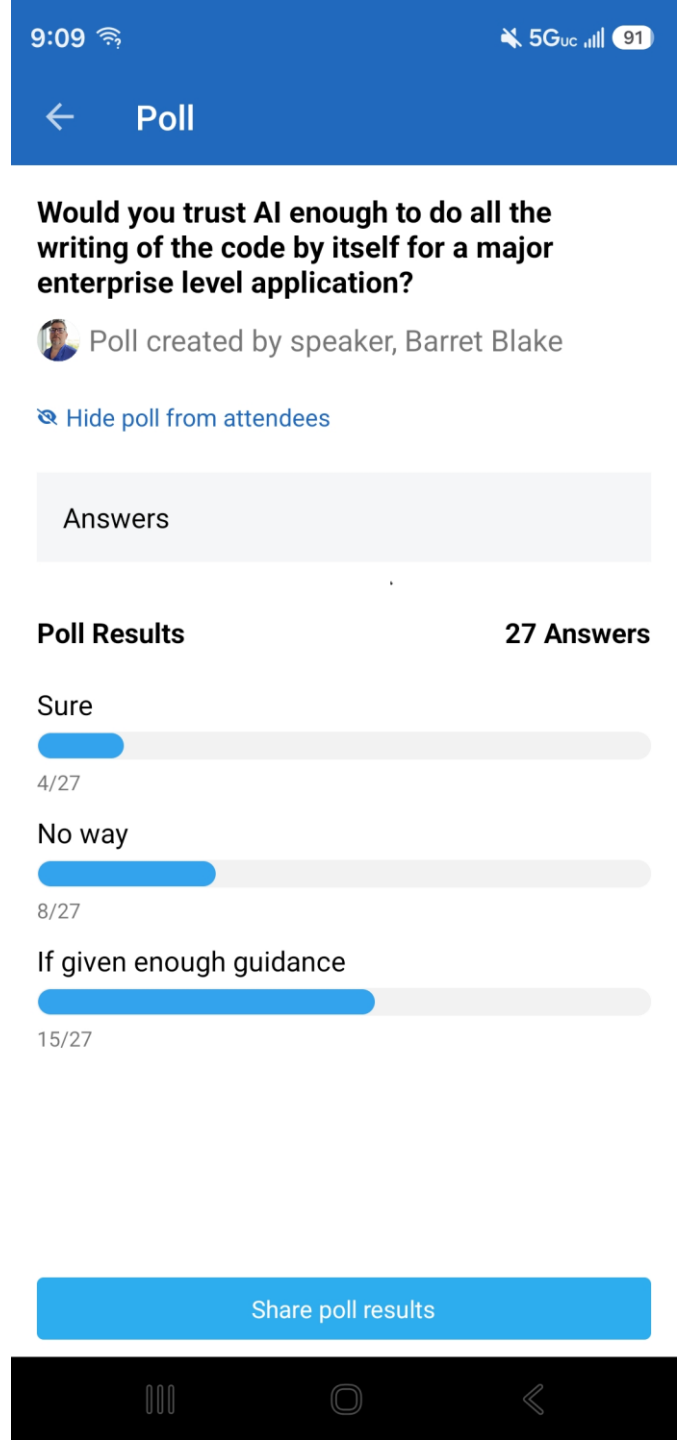
Architect · NimblePros

Microsoft® Most Valuable Professional

A few things about me

- Microsoft MVP — .NET
- Husband & Father
- Coder, Blogger & Speaker
- Mentor
- Gamer
- Model Railroader & LEGO Enthusiast
- Buckeyes & Browns Fan

NimblePros
Better software development. Delivered.


9:09
5G
91
Poll
Would you trust AI enough to do all the writing of the code by itself for a major enterprise level application?
Poll created by speaker, Barret Blake
Hide poll from attendees
Answers
Poll Results
27 Answers
Sure
4/27
No way
8/27
If given enough guidance
15/27
Share poll results

THE PROBLEM

Vibe Coding: Staring at a Blank Page



What is vibe coding?

Tossing vague prompts at an AI coding assistant and accepting whatever it hands back — no plan, no spec, just vibes.

Like starting a novel with no outline, no characters, no ending in mind.

The result:

- Unpredictable, inconsistent code quality
- Features that miss the actual requirement
- Hours lost debugging the AI's misunderstandings
- Over-engineered solutions nobody asked for

WHY IT HAPPENS

We Treat AI Like a Search Engine

“The issue isn't the coding agent's ability, but our approach. We treat coding agents like search engines when we should treat them more like literal-minded pair programmers.”

— Den Delimarsky, former GitHub Principal Product Manager

The key insight

AI assistants need clear, structured context and specifications — not vague wishes. They excel at execution when you give them real direction.

THE SHIFT

What Is Spec-Driven Development?

An approach where specifications become the source of truth — guiding AI agents to build exactly what's needed through clear, structured, living requirements.

Specifications First

Define what you want before a line of code exists

Living Documents

Specs evolve continuously — not written once and forgotten

Human Review

Every phase needs explicit approval before moving on

Single Source of Truth

One spec guides both humans and AI agents

Docs Live With Code

Specs and implementation share the same repo

THE TOOLKIT

GitHub Spec Kit

An open-source toolkit that brings Spec-Driven Development to life — a simple CLI plus structured templates that work with the agent you already use.

106K+	30	105	200+
GitHub stars	Agent integrations	Community extensions	Contributors

- Open source & agent-agnostic by design
- A Python CLI plus battle-tested templates
- Drops into VS Code or your favorite AI tool

First released Sept 2025 by GitHub — now a fast-moving community platform.

THE WORKFLOW

From Intent to Implementation

Seven steps take InkWell from a single sentence to shipped code:



Quality gates along the way: `/speckit-clarify` before planning, `/speckit-checklist` and `/speckit-analyze` before you implement.

OUR RUNNING DEMO

Meet InkWell

A novelist's writing studio — the app we'll build end-to-end with Spec Kit.



Chapter drafting

Distraction-free editor with autosave

Character & plot tracker

Keep your cast and threads straight

Word-count goals

Daily targets and streaks that motivate

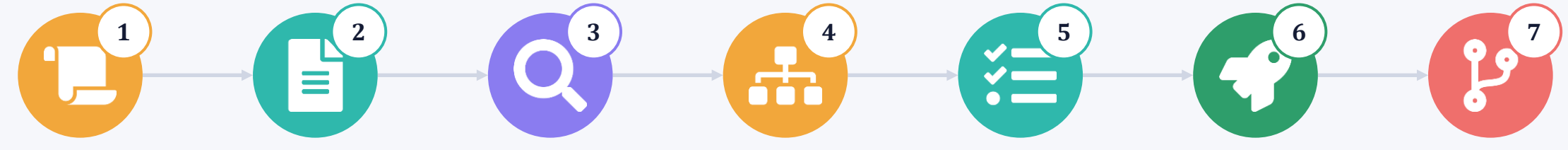
Local-first & private

Manuscripts stay on the writer's device

Same story, every phase: watch InkWell grow from a sentence into a working app.

Specification 2 – Adding reading time

From one sentence to a reviewed plan



Step by step through the process.

STEP 1 · /SPECKIT-CONSTITUTION

Set the Ground Rules

```
/speckit-constitution
```



The foundation

A set of non-negotiable principles the agent must honor in every later phase — quality bars, tech constraints, UX and accessibility standards. Stored in .specify/memory/constitution.md.



InkWell's constitution (9 principles)

- Clean architecture — dependencies point inward, always
- Tests before implementation, for every feature
- WCAG 2.1 AA is a mandate, not an aspiration
- Local-first, encrypted at rest; nothing leaves without consent

STEP 2 · /SPECKIT-SPECIFY

Describe What to Build

```
/speckit-specify
```



The what & the why

Focus on outcomes, not tech. Capture goals, target users, core features, and success criteria. Spec Kit creates a feature branch (001-manuscript-drafting) and a spec.md full of user stories.



InkWell's spec prompt

“Build an app for novelists to draft chapters and organize them into a manuscript. Writers track characters and plot threads, set daily word-count goals, and write in a distraction-free editor. Everything works offline and saves locally.”

STEP 3 · /SPECKIT-CLARIFY

```
/speckit-clarify
```

Close the Gaps Before You Plan

Structured Q&A

A sequential, coverage-based interview surfaces what the first draft missed and records answers in a Clarifications section. Run it before /speckit-plan to cut rework. (Formerly /quizme.)

What InkWell actually got asked

- How are inline images stored, given encrypted local-first?
- How does the editor present content in distraction-free mode?
- What counts as a word — prose only, or markdown syntax too?
- Export granularity: whole manuscript, per chapter, or both?

STEP 4 · /SPECKIT-PLAN

Choose the How

```
/speckit-plan
```



The technical plan

Now name your stack. The agent reads the spec + constitution and produces plan.md, research.md, data-model.md, contracts/, and quickstart.md — architecture, data, and APIs in detail.



InkWell's plan prompt

“Use .NET MAUI with a Clean Architecture design and a minimal library footprint. Store manuscripts in an encrypted local database — nothing is uploaded. Keep the editor framework-agnostic and fully keyboard-navigable.” → the agent chose SQLite + SQLCipher (AES-256), key in SecureStorage. You never named it.

THE CRAFT

Wishes vs. Specs: Writing for a Literal-Minded Reader

A WISH	A SPEC · WORD FOR WORD FROM INKWELL'S spec.md	
✗ *"The editor should be fast."*	✓ Enter and exit distraction-free mode, back at the exact cursor position, in under 1 second.	SC-006
✗ *"Word counts should be accurate."*	✓ Matches an independent prose-word count — excluding markdown syntax and image markers — 100% of the time.	SC-005
✗ *"It works offline."*	✓ Every capability works with networking fully disabled; 100% of user stories complete offline.	SC-002
✗ *"Never lose my work."*	✓ No edit lost beyond the last few seconds of typing; restored on reopen in 100% of test cases.	SC-003



The test: could two reasonable people disagree about whether it passed?

My own SC-004 says typing must be "perceived as instantaneous." The plan quietly made it 16 ms. **Leave a wish in the spec and the agent picks the number for you.**

STEP 5 · /SPECKIT-TASKS

```
/speckit-tasks
```

Break It Into Actionable Work



From plan to a build list

- Tasks grouped by user story — each ships independently
- Ordered by dependency (models → services → endpoints)
- Parallel-safe work flagged with [P]
- Exact file paths on every task
- Test-first ordering when TDD is requested



Bonus: /speckit-taskstoissues

Turn the generated task list into tracked GitHub issues in one command — so the whole team can pick up InkWell's work.

```
tasks.md
  [P] T003  models/manuscript.ts
  [P] T004  models/chapter.ts
      T007  editor/autosave.ts
      T012  export/epub.ts
```

STEP 6 · /SPECKIT-IMPLEMENT

```
/speckit-implement
```

Let the Agent Build InkWell

Validates prerequisites

Constitution, spec, plan, and tasks all present

Parses tasks.md

Reads the ordered breakdown and dependencies

Executes in order

Respects sequence and [P] parallel markers

Follows TDD & the plan

Tests first where you asked; principles enforced

The result: a working InkWell — chapters, characters, goals, and a distraction-free editor — built from specs, not vibes.

STEP 7 · REVIEW, VERIFY, MERGE

Reading the Diff: Five Smells

1. **Code that contradicts a numbered requirement** — REAL · INKWELL
   FR-004: “without requiring an explicit save action.” Three screens shipped with Save buttons.

2. **Green tests, broken app** — REAL · INKWELL
   Chapter text never saved. The tests substitute the editor — so every one of them passed.

3. **Tests that changed with the code**
   An assertion loosened in the same diff as the fix it is supposed to verify.

4. **A gate that graded itself**
   Constitution Check: 9 of 9 PASS — written by the author of the plan it is passing.

5. **Work outside the lines**
   Files the task never named. A package the plan never chose.

Smell #2, in three lines

```
function reportContent(type) {
  if (!view || !chapterId)
    return;  // every keystroke
}
```

chapterId was only ever set by a message that got dropped on the floor. No error. No failing test. No saved chapters.

`specs/001-…/research.md  §5.4`

NEW IN 2026

Make Spec Kit Your Own

Extensions

ADD new capabilities

New commands, phases, and integrations. CI Guard, Architecture Guard, Jira sync — or an InkWell “manuscript-export” command.

```
specify extension add <name>
```

Presets

CHANGE how it works

Reshape templates, terminology, and standards without new commands. Enforce a house style guide on every spec and plan.

```
specify preset add <name>
```

Or swap the whole process: AIDE, Canon, Product Forge, MAQA — community presets that redefine SDD end-to-end. 105 extensions, 22 presets, and counting.

When NOT to Use Spec-Driven Development



Throwaway work

Run-once scripts, data fixes, disposable spikes

Genuine exploration

When you don't know what you want, a spec is a lie

Very small, known changes

Adding a field. The overhead exceeds the risk removed.

Nobody reading the output

Rubber-stamping is vibe coding with extra latency

Small and disposable, vibe it. Unknown and exploratory, prototype it. Real, shared, outliving the sprint? Spec it.

IN SUMMARY

Five Things to Remember

1	**Vibe coding doesn't scale**	Clear specs are essential for reliable AI-assisted development
2	**Spec Kit gives you structure**	A proven path from constitution to implementation — and beyond
3	**It's more powerful in 2026**	Extensions, presets, and skills mode make it truly yours
4	**Works with your tools**	30+ agents, no lock-in — Copilot, Claude, Cursor, and more
5	**You're still the author**	Start small, iterate often, and keep a human in the loop

Thank You!

Questions?

Repo & Slides: `github.com/barretb/inkwell`



GitHub Spec-kit `github.com/github/spec-kit`

BlueSky @barretblake.dev

Web barretblake.dev

YouTube youtube.com/@barretcodes

LinkedIn linkedin.com/in/barretblake

NimblePros blog.nimblepros.com

